

04 - AI Collaboration Guide

Project: Dragon Merchant Guild (*Working Title*)

Document Version: 1.0

Status: Active

1. Purpose

Artificial Intelligence is used to **accelerate learning**, not replace it.

The goal of this project is to become an independent C# developer.

Therefore, AI acts as a mentor, reviewer, and advisor rather than a programmer.

2. Guiding Philosophy

The player writes the game.

The AI teaches the player.

Every interaction with AI should increase understanding.

If an interaction removes the need to think, it should usually be avoided.

3. Primary Rule

Before asking AI for help, genuinely attempt the problem yourself.

Getting stuck is expected.

Skipping the thinking process is not.

4. Roles of AI

During this project, AI has several clearly defined roles.

Teacher

Explains concepts.

Examples:

- What is boxing?
- Why use `ref`?
- What does this compiler warning mean?
- How does LINQ work?

The teacher focuses on understanding rather than implementation.

Reviewer

Reviews completed code.

Typical questions include:

- Is this idiomatic C#?
- Is this readable?
- Would you name this differently?
- Does this follow SOLID naturally?
- What edge cases am I missing?

The reviewer critiques the code without rewriting it.

Challenger

Tests understanding.

Rather than providing answers, the challenger asks questions.

Examples:

- Why did you choose a class instead of a struct?
- Could this method return a tuple?
- What happens if this value is null?

- Why is this parameter `ref` instead of `in`?
- What assumptions are you making?

The goal is to strengthen reasoning.

Mentor

Provides long-term guidance.

Examples:

- Prioritizing features
- Avoiding overengineering
- Identifying technical debt
- Suggesting refactoring opportunities
- Planning future improvements

The mentor focuses on project direction rather than syntax.

Worldbuilding Advisor

Provides feedback on dragons, lore, economy, contracts, and progression.

The advisor should expand ideas without taking ownership of the world's creativity.

Original dragon designs remain the developer's creations.

5. Roles AI Should Avoid

Unless explicitly requested, AI should avoid:

- writing production code
- implementing entire systems
- solving assignments immediately
- designing everything from scratch
- making architectural decisions without discussion

These activities reduce learning.

6. The Learning Workflow

Every study session follows the same process.

Step 1 — Study

Read the curriculum.

Take notes.

Understand the concept.

Do not begin coding immediately.

Step 2 — Build

Close AI.

Implement the feature independently.

Experiment.

Make mistakes.

Debug.

Think.

Step 3 — Play

Run the game.

Interact with the new feature.

Observe its behavior.

Step 4 — Review

Only after completing an honest attempt should AI become involved.

Possible prompts include:

- Review my implementation.
 - Ask me questions about this code.
 - Find possible bugs without fixing them.
 - Suggest improvements.
 - Challenge my assumptions.
-

Step 5 — Reflect

Answer questions such as:

What did I learn?

What surprised me?

What would I redesign?

What became easier?

Reflection is considered part of learning.

7. Preferred Questions

Good questions encourage reasoning.

Examples include:

Why is this better?

What are the trade-offs?

Can this fail?

What assumptions am I making?

How would this scale?

What edge cases exist?

What design pattern naturally appears here?

8. Questions To Avoid

Questions that bypass thinking should generally be avoided.

Examples:

Write this feature.

Build this class.

Implement my inventory.

Finish this assignment.

Solve this bug without explanation.

Generate the entire project.

These are acceptable only after significant personal effort or when intentionally studying an alternative implementation.

9. Code Reviews

Preferred review style:

First:

Highlight strengths.

Second:

Identify problems.

Third:

Ask questions.

Finally:

Suggest improvements.

The goal is understanding rather than simply correcting mistakes.

10. Bug Assistance

When debugging, AI should not immediately provide the answer.

Preferred order:

1. Ask diagnostic questions.
2. Explain likely causes.
3. Suggest investigation steps.
4. Reveal the solution only if needed.

This mirrors how an experienced mentor teaches.

11. Project Discussions

Design discussions should focus on trade-offs rather than correctness.

Example:

Instead of:

"This is wrong."

Prefer:

"This approach is simpler now, but becomes harder to maintain when..."

The developer should make the final decision.

12. Creativity

Original ideas belong to the developer.

AI may:

- critique ideas
- expand ideas

- identify inconsistencies
- ask questions

AI should avoid replacing the developer's creativity.

The world should remain recognizably the developer's imagination.

13. Measuring Progress

Progress is **not** measured by:

- lines of code
- completed files
- number of dragons
- number of features

Progress is measured by:

- better reasoning
 - cleaner architecture
 - improved debugging
 - stronger understanding
 - greater confidence
-

14. When AI May Write Code

There are situations where requesting implementation is appropriate.

Examples include:

- comparing approaches after finishing your own
- learning an unfamiliar API
- studying advanced C# idioms
- reviewing professional practices
- exploring optimizations
- generating disposable prototypes

Even then, generated code should be studied rather than copied blindly.

15. The "Teacher First" Principle

Whenever possible, AI should respond in this order:

1. Ask questions.
 2. Encourage reasoning.
 3. Explain concepts.
 4. Review existing work.
 5. Demonstrate with examples.
 6. Write complete implementations only when explicitly requested.
-

16. AI Prompt Templates

These prompts may be adapted for any AI assistant.

Concept Tutor

"I am currently studying <topic>. Explain it without solving future exercises for me. Compare it to C whenever helpful, and point out common misconceptions."

Code Reviewer

"I wrote this feature myself. Please review it like a senior developer. Do not rewrite it. Point out strengths, weaknesses, possible bugs, code smells, and edge cases. Ask me questions that test my understanding."

Challenger

"I believe my implementation is correct. Challenge my assumptions. Ask difficult questions that might expose weaknesses in my reasoning."

Design Mentor

"I am considering two designs. Help me compare the trade-offs without choosing for me unless one option is clearly superior."

Debugging Mentor

"My code is not behaving as expected. Do not give me the solution immediately. Help me debug it by asking questions and suggesting investigation steps."

Dragon Advisor

"I designed this dragon. Evaluate whether it fits naturally within the Dragon Merchant Guild world. Suggest improvements while preserving the original concept."

17. Final Reminder

The objective of this project is not to finish a game.

The objective is to become the kind of developer who no longer needs tutorials for every new idea.

The game is simply the path chosen to reach that destination.

Personal Rule

If AI thinks more than I do, I am using it incorrectly.

The ideal study session ends with greater understanding—not merely more code.